

Curso de Go

Aula 17 - Request/Response + JSON

Professor : Antônio Marcelo

NÚCLEA
ACADEMY

 **C/B** [código_] **BRAZUCA** [3]





Nessa Aula nos Vamos Ver

- **Manipulação de requests HTTP**
- **Criação de responses HTTP**
- **Leitura de dados enviados pelo cliente**
- **Retorno de dados em JSON**
- **Uso de json.Marshal**
- **Uso de json.Unmarshal**
- **Headers HTTP**
- **Status codes**
- **Estrutura básica de uma API REST**
- **Parte Prática**



O que é uma request HTTP?

- Uma request é a mensagem enviada pelo cliente para o servidor.
- Exemplos de clientes:
 - Navegador
 - Aplicativo mobile
 - Frontend em React
 - Postman
 - Outro backend
- Uma request normalmente possui:
 - Método HTTP
 - URL
 - Headers
 - Body
 - Query parameters

Exemplo conceitual:

```
POST /produto HTTP/1.1
Content-Type: application/json
```

```
{
  "nome": "Teclado",
  "preco": 150.50
}
```



O que é uma request HTTP?

- Em Go, uma request é representada por:

```
*r http.Request
```

- Exemplo de handler:

```
func handler(w http.ResponseWriter, r
*http.Request) {
    fmt.Println(r.Method)
    fmt.Println(r.URL.Path)
}
```



O que é uma response HTTP?

- Uma response é a resposta que o servidor envia para o cliente.
- Ela normalmente possui:
 - Status code
 - Headers
 - Body

Exemplo conceitual:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "mensagem": "Produto encontrado"
}
```



```
func helloHandler(w
http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusOK)
    w.Write([]byte("Olá, mundo!"))
}
```

Em Go, usamos:

```
w http.ResponseWriter
```




Criando um servidor HTTP básico

```
package main

import (
    "fmt"
    "net/http"
)

func helloHandler(w http.ResponseWriter, r
*http.Request) {
    fmt.Fprintln(w, "Olá! Esta é uma API em
Go.")
}

func main() {
    http.HandleFunc("/hello", helloHandler)

    fmt.Println("Servidor rodando em
http://localhost:8080")
    http.ListenAndServe(":8080", nil)
}
```



- **Explicação:**

- **http.HandleFunc** registra uma rota
- **/hello** é o caminho da rota
- **helloHandler** é a função que será executada
- **w** representa a response
- **r** representa a request
- **http.ListenAndServe** inicia o servidor
- **Teste no navegador:**

`http://localhost:8080/hello`



Lendo informações da request

- Podemos acessar várias informações da request.
- Exemplo:

```
func infoHandler(w http.ResponseWriter, r *http.Request)
{
    fmt.Fprintf(w, "Método: %s\n", r.Method)
    fmt.Fprintf(w, "Caminho: %s\n", r.URL.Path)
    fmt.Fprintf(w, "Query: %s\n", r.URL.RawQuery)
}
```

- Registrando a rota:

```
http.HandleFunc("/info", infoHandler)
```

- Teste:

```
http.HandleFunc("/info", infoHandler)
```




Lendo informações da request

- Para ler query parameters:

```
nome := r.URL.Query().Get("nome")
idade := r.URL.Query().Get("idade")
```

- Exemplo:

```
func usuarioHandler(w http.ResponseWriter, r
*http.Request) {
    nome := r.URL.Query().Get("nome")

    if nome == "" {
        nome = "visitante"
    }

    fmt.Fprintf(w, "Olá, %s!", nome)
}
```




Trabalhando com JSON em Go

- JSON é um formato muito usado para troca de dados entre sistemas.
- Exemplo de JSON:

```
{  
  "id": 1,  
  "nome": "Notebook",  
  "preco": 3500.90  
}
```

Em Go, representamos esse dado com struct:

```
type Produto struct {  
  ID int `json:"id"`  
  Nome string `json:"nome"`  
  Preço float64 `json:"preco"`  
}
```



Trabalhando com JSON em Go

- As tags json definem o nome dos campos no JSON.
- Sem as tags, o Go usaria os nomes da struct:

```
{  
  "ID": 1,  
  "Nome": "Notebook",  
  "Preco": 3500.90  
}
```

Com tags, o JSON fica mais adequado para APIs:

```
{  
  "id": 1,  
  "nome": "Notebook",  
  "preco": 3500.90  
}
```



Convertendo struct para JSON com Marshal

- **json.Marshal** transforma uma struct em JSON.
- **Exemplo:**

```
package main

import (
    "encoding/json"
    "fmt"
)

type Produto struct {
    ID      int      `json:"id"`
    Nome    string   `json:"nome"`
    Preço   float64  `json:"preco"`
}

func main() {
    produto := Produto{
        ID:      1,
        Nome:    "Mouse Gamer",
        Preço:   199.90,
    }

    jsonData, err := json.Marshal(produto)
    if err != nil {
        fmt.Println("Erro ao gerar JSON:", err)
        return
    }

    fmt.Println(string(jsonData))
}
```



- **Resultado:**

```
{"id":1,"nome":"Mouse Gamer","preco":199.9}
```

- **Uso comum:**
 - Retornar dados de uma **API**
 - Salvar dados em arquivo
 - Enviar dados para outro sistema



Retornando JSON em uma API

- Exemplo de endpoint que retorna um produto em JSON:

```
func produtoHandler(w http.ResponseWriter, r
*http.Request) {
    produto := Produto{
        ID:      1,
        Nome:    "Teclado Mecânico",
        Preco:   350.00,
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusOK)

    json.NewEncoder(w).Encode(produto)
}
```



Retornando JSON em uma API

- Servidor completo:



```
package main

import (
    "encoding/json"
    "fmt"
    "net/http"
)

type Produto struct {
    ID      int      `json:"id"`
    Nome    string   `json:"nome"`
    Preço   float64  `json:"preco"`
}

func produtoHandler(w http.ResponseWriter, r *http.Request) {
    produto := Produto{
        ID:      1,
        Nome:    "Teclado Mecânico",
        Preço:   350.00,
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusOK)

    json.NewEncoder(w).Encode(produto)
}

func main() {
    http.HandleFunc("/produto", produtoHandler)

    fmt.Println("Servidor rodando na porta 8080")
    http.ListenAndServe(":8080", nil)
}
```




Convertendo JSON para struct com Unmarshal

- **json.Unmarshal** transforma JSON em struct.
- **Exemplo:**

```
package main

import (
    "encoding/json"
    "fmt"
)

type Produto struct {
    ID      int      `json:"id"`
    Nome    string   `json:"nome"`
    Preço   float64  `json:"preco"`
}

func main() {
    dados :=
[]byte(`{"id":2,"nome":"Monitor","preco":899.90}`)

    var produto Produto

    err := json.Unmarshal(dados, &produto)
    if err != nil {
        fmt.Println("Erro ao ler JSON:", err)
        return
    }

    fmt.Println(produto.ID)
    fmt.Println(produto.Nome)
    fmt.Println(produto.Preco)
}
```



- **Resultado:**

```
2
Monitor
899.9
```

- **Importante:**
 - **Marshal** transforma Go em JSON
 - **Unmarshal** transforma JSON em Go
 - No Unmarshal, passamos o endereço da variável com &



Recebendo JSON via POST

- Agora vamos criar uma rota que recebe um produto via POST.
- Exemplo:

```
func criarProdutoHandler(w http.ResponseWriter, r
*http.Request) {
    if r.Method != http.MethodPost {
        w.WriteHeader(http.StatusMethodNotAllowed)
        json.NewEncoder(w).Encode(map[string]string{
            "erro": "Método não permitido",
        })
        return
    }

    var produto Produto

    err := json.NewDecoder(r.Body).Decode(&produto)
    if err != nil {
        w.WriteHeader(http.StatusBadRequest)
        json.NewEncoder(w).Encode(map[string]string{
            "erro": "JSON inválido",
        })
        return
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusCreated)

    json.NewEncoder(w).Encode(map[string]interface{}{
        "mensagem": "Produto criado com sucesso",
        "produto":  produto,
    })
}
```





Recebendo JSON via POST

- **Exemplo de request:**

```
POST /produto HTTP/1.1
Content-Type: application/json
```

```
{
  "id": 10,
  "nome": "Cadeira Gamer",
  "preco": 1200.00
}
```

- **Resposta esperada:**

```
{
  "mensagem": "Produto criado com sucesso",
  "produto": {
    "id": 10,
    "nome": "Cadeira Gamer",
    "preco": 1200
  }
}
```



Headers, status codes e boas práticas

- Headers são metadados da request ou response.
- Exemplo de header importante:

```
w.Header().Set("Content-Type", "application/json")
```

- Status codes comuns:

```
http.StatusOK           // 200
http.StatusCreated      // 201
http.StatusBadRequest  // 400
http.StatusNotFound     // 404
http.StatusMethodNotAllowed // 405
http.StatusInternalServerError // 500
```

- Exemplo de resposta padronizada de erro:

```
func erroJSON(w http.ResponseWriter, status
int, mensagem string) {
    w.Header().Set("Content-Type",
"application/json")
    w.WriteHeader(status)

    json.NewEncoder(w).Encode(map[string]string{
        "erro": mensagem,
    })
}
```



- Uso:

```
erroJSON(w,
http.StatusBadRequest, "Dados
inválidos")
```



Boas práticas

- **Sempre defina Content-Type ao retornar JSON**
- **Use status codes corretos**
- **Valide o método HTTP**
- **Trate erros de Decode e Encode**
- **Não ignore o erro retornado pelas funções**
- **Evite retornar texto solto em APIs JSON**
- **Padronize respostas de erro**
- **Separe handlers em funções pequenas**
- **Use structs para organizar os dados**



Boas práticas

- Exemplo final de rota organizada:

```
func listarProdutoHandler(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodGet {
        erroJSON(w, http.StatusMethodNotAllowed, "Use o método GET")
        return
    }

    produto := Produto{
        ID:      1,
        Nome:    "Notebook",
        Preco:   4500.00,
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusOK)

    json.NewEncoder(w).Encode(produto)
}
```

The screenshot shows a Go IDE with a file named 'Untitled-1' in the 'package main' directory. The code defines a struct 'rect' with 'width' and 'height' of type 'int'. It includes two methods: 'area()' which returns the area, and 'perim()' which returns the perimeter. The 'main' function creates a 'rect' with width 10 and height 5, prints its area and perimeter, and then demonstrates calling these methods on a pointer to the 'rect'.

```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de receptor
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

NÚCLEA
ACADEMY

Parte Prática

Request/Response + JSON